Presenter Guide

GTSRB Traffic Sign Recognition Project

For a presenter with basic machine learning knowledge

How to Use This Guide

This document is both a learning guide and a speaking script. Read the explanation before each section, then use the Presenter wording paragraph when explaining the project. You do not need to memorize every technical term. The main story is simple: we verified images, understood them, trained models, measured their mistakes, and packaged the result.

Suggested duration	Sections	Main message
5 minutes	1–4	What the problem and data are
10 minutes	5–8	How the models learn and how results are measured
5 minutes	9–11	Results, limitations, and live demo

1. One Minute Project Summary

The project recognizes a traffic sign from a cropped image. It uses the German Traffic Sign Recognition Benchmark, which has 43 sign categories. The pipeline downloads and verifies the dataset, explores its properties, resizes and normalizes images, creates a leakage-aware validation split, trains two neural networks, evaluates them on the official test set, and predicts a class for a new image.

Presenter wording: "Our objective is not to detect where a sign is in a full road scene. Our objective is classification: given one cropped sign image, predict which of 43 known classes it belongs to."

Term	Plain-English meaning
Image classification	Choose one label for an image
Training	Let a model learn patterns from labeled examples
Validation	Use held-out training examples to guide decisions
Test set	Final unseen examples used for reporting
Inference	Use a trained model to predict a new image

2. The Whole Pipeline

Think of the project as a factory line. Each stage produces something that the next stage needs. If the images are wrong, the analysis is wrong. If the split leaks near-duplicates, the score looks artificially good. If evaluation is incomplete, we do not know which classes fail.

Stage	Input	Output	Why it exists
Acquire	Zenodo archive	Verified local files	Trust the source
Explore	Images and labels	EDA graphs	Understand variation
Prepare	Raw images	Normalized datasets	Create consistent model input
Train	Training batches	Saved models	Learn visual patterns

Evaluate	Official test images	Metrics and plots	Measure generalization
Infer	One new image	Class and confidence	Use the model

Presenter wording: "The important point is that the model is only one part of the system. A machine-learning project is credible when the complete path from data to result is reproducible and measurable."

3. Dataset Acquisition and Integrity

The full GTSRB classification archive is downloaded from Zenodo. It contains 39,209 labeled training images, 12,630 official test images, and 43 class IDs from 0 through 42. The downloader calculates the MD5 hash of data.zip and compares it with the published value 7faafd0d78d831c7eb4d555 641a83a62 before extracting anything.

Why use a checksum? A download can be interrupted or corrupted while still producing a file. A checksum is a compact fingerprint of the file. If even a small part changes, the computed fingerprint should change, so extraction is stopped instead of silently using bad data.

Run: python scripts/download_dataset.py

- The script skips an already valid archive, so repeated runs are safe.
- It reports a clear error for a checksum mismatch or incomplete extraction.
- It checks that all expected classes 0–42 are present.
- Dataset and trained artifacts are excluded from Git because they are large generated files.

Presenter wording: "Before training, we prove that we have the intended archive. This makes the experiment repeatable on another machine."

4. Exploratory Data Analysis

EDA means exploratory data analysis. It is the stage where we ask what the data looks like before choosing or trusting a model. The project creates these graphs with python scripts/run_eda.py.

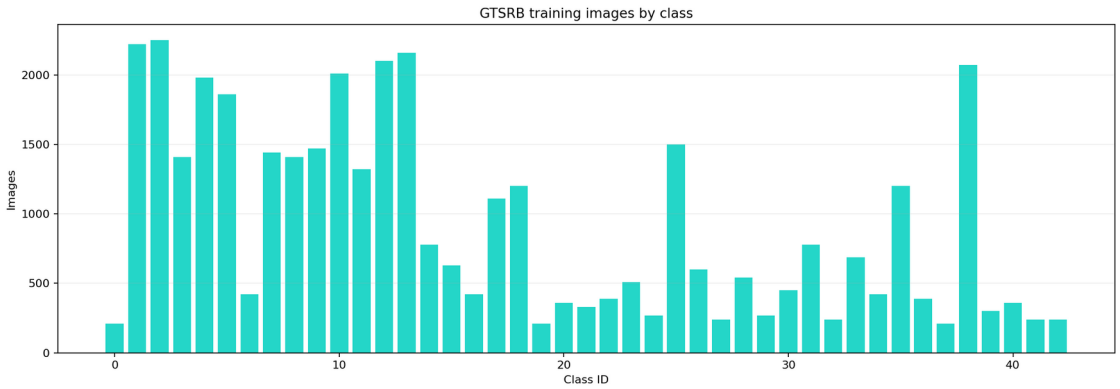


Figure 1. Training examples per class.

How to read it: the horizontal axis is the class ID and the vertical axis is the number of training images. Some classes have around 210 examples while others have more than 2,000. This is class imbalance. A model can achieve a reasonable overall accuracy while performing poorly on smaller classes, so we also report macro F1.

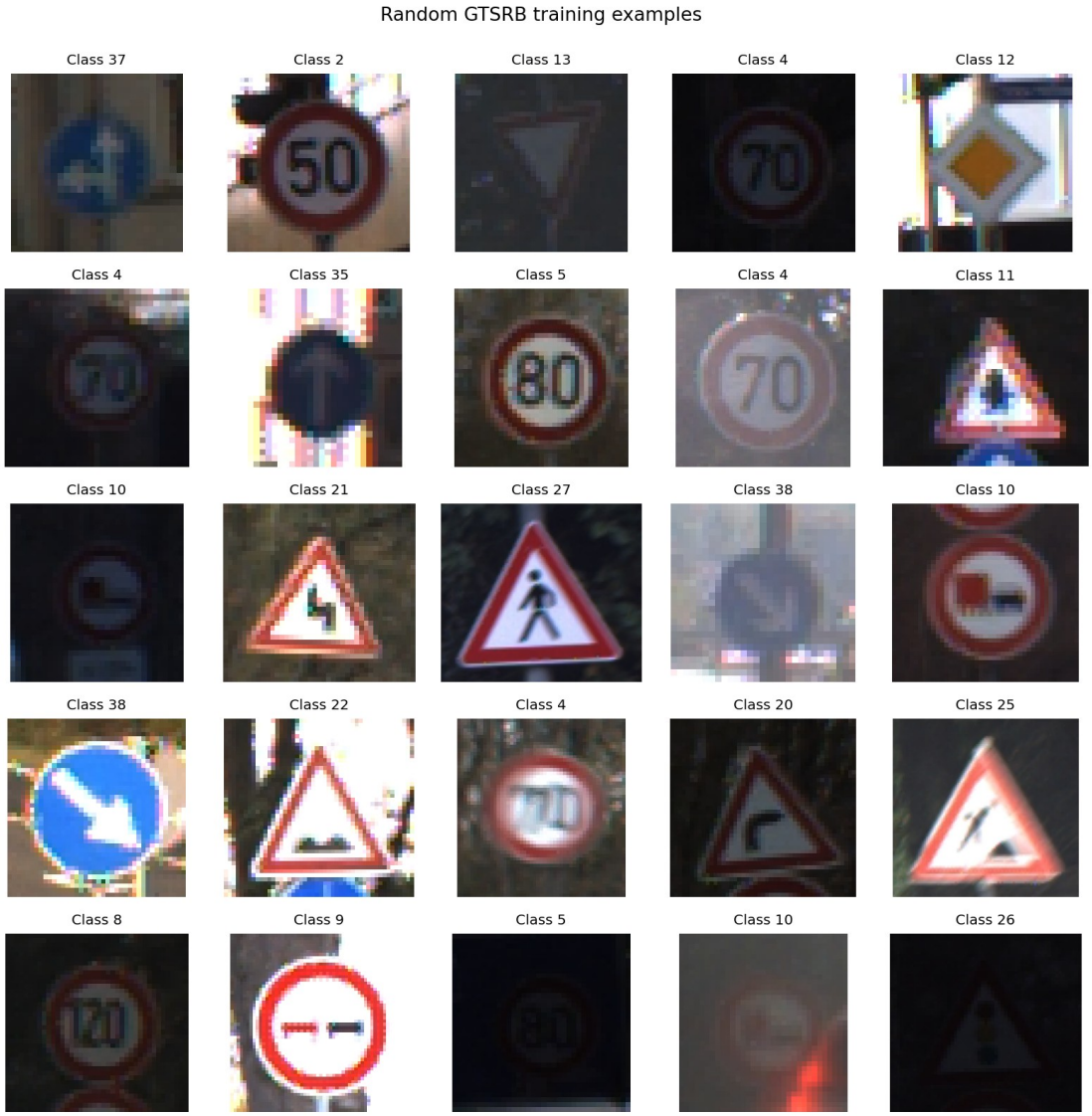


Figure 2. Representative traffic-sign images.

How to read it: the grid shows the visual variety the model must handle. Signs can be dark, bright, blurred, tilted, partially obstructed, or photographed against different backgrounds. Similar-looking signs make the task harder.

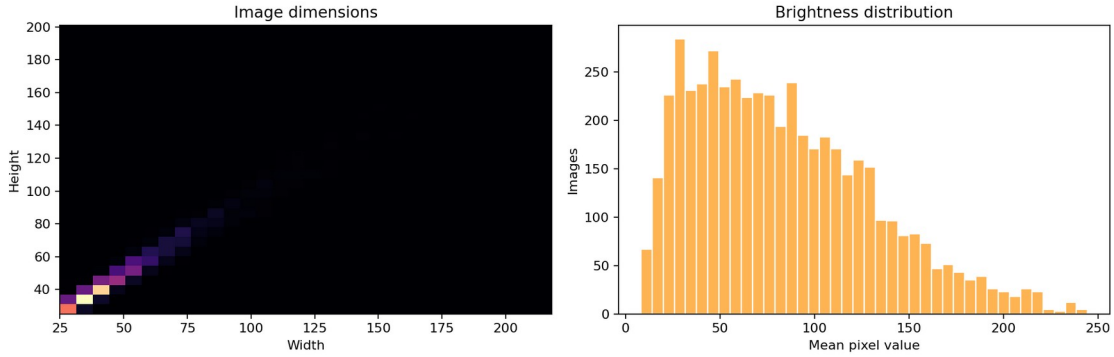


Figure 3. Sampled image dimensions and brightness.

How to read it: the left plot shows that source image width and height are not identical. The right plot shows a broad brightness distribution. Therefore, the model receives resized images and augmentation that teaches it not to depend on one exact size or lighting condition.

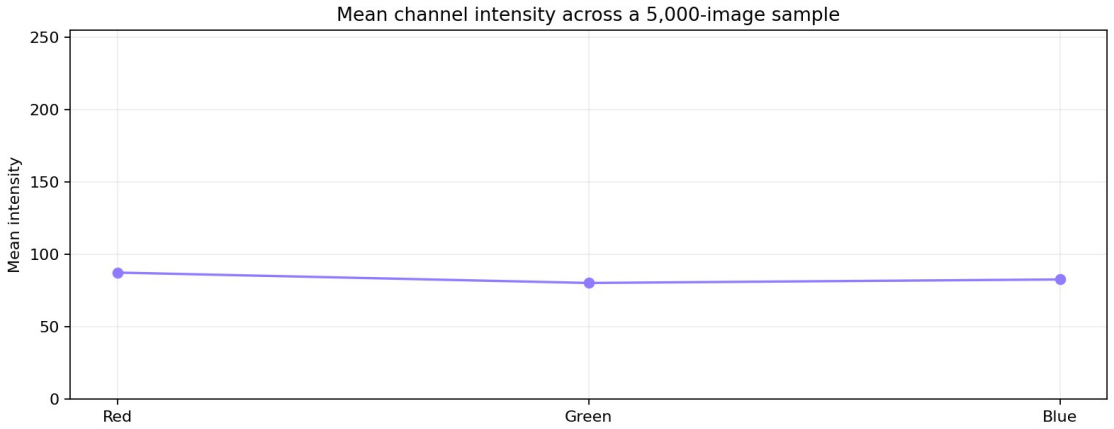


Figure 4. Mean red, green, and blue intensity.

The RGB graph summarizes average channel intensity over a sample. It is not a class predictor by itself. It is a sanity check that color and illumination vary in the dataset and that the preprocessing pipeline should preserve useful color information.

5. Labels, Splits, and Leakage

A label is the correct answer attached to an image. Training images get their labels from their class directories. Test labels are read from Test.csv. The project keeps the official GTSRB test split untouched until final evaluation.

The training data is divided into a training subset and a validation subset. The model learns from the training subset; validation examples are used to monitor whether learning is improving on data it did not directly optimize against.

Leakage happens when information from the evaluation side accidentally appears in the training side. In image datasets, related frames can be nearly identical. This project groups related image sequences before splitting, which makes validation more honest.

Presenter wording: "Validation helps us choose when to stop and which checkpoint is best. The official test set is not used for those decisions; it is reserved for the final score."

6. Preprocessing and Augmentation

Operation	What happens	Why
Resize	Convert images to 64×64 or 128×128	Neural networks need consistent tensor shapes
Normalize	Scale pixel values from 0–255 to 0–1	Stable numerical optimization
Rotate	Small random angle	Handle tilted signs
Translate	Small horizontal or vertical shift	Handle imperfect framing
Zoom	Small random scale change	Handle distance variation
Brightness and contrast	Moderate lighting changes	Improve robustness

A pixel is a number representing color intensity. A color image is stored as height × width × 3 because it has red, green, and blue channels. Normalization changes a pixel such as 128 into approximately 0.502. The visual meaning stays the same, but the numbers are easier for the optimizer to work with.

Augmentation creates a changed version of a training image without changing its label. A slightly rotated speed-limit sign is still a speed-limit sign. Augmentation is applied during training, not to the official test images.

7. Model One Custom CNN

CNN means convolutional neural network. A convolutional layer uses small learnable filters that slide across an image. Early filters often learn simple edges and colors. Deeper layers combine those patterns into shapes and symbols. Pooling reduces spatial size, dropout reduces over-reliance on individual units, and the final dense layer produces 43 class scores.

The custom CNN is the baseline. A baseline is a reference model: it tells us what performance a straightforward architecture can achieve. It is compact and easier to explain, inspect, and run on CPU.

Presenter wording: "The custom CNN learns its visual features from this dataset. Its advantage is transparency and low complexity; its limitation is that it starts without broad visual knowledge."

8. Model Two MobileNetV2

MobileNetV2 is a lightweight CNN designed to use fewer computations than many large networks. Transfer learning means starting from a model that has already learned general visual features, then adapting it to our 43 traffic-sign classes. The project also supports no-pretrained mode for offline smoke tests.

The comparison is useful because the two models learn in different ways: the custom CNN learns from scratch, while MobileNetV2 can reuse general edge, texture, and shape features. In a final benchmark, both models should receive matched training budgets.

9. Training and Callbacks

Training happens in batches. One batch is a small group of images processed together. An epoch means the model has seen the training data once. During each batch, the model predicts, compares its prediction with the true label, calculates a loss, and adjusts its weights using gradient descent.

Callback	Beginner explanation
Early stopping	Stop if validation performance stops improving
Learning-rate reduction	Make smaller updates when progress plateaus
Best checkpoint	Save the version with the best validation result
Deterministic seed	Make splits and random operations repeatable as far as hardware allows

Loss is an error value that training tries to minimize. Accuracy is the fraction of correct predictions. They are related but not identical: a model can improve its confidence and loss before its hard class decision changes.

10. Evaluation Metrics

Metric	Formula idea	Question answered
Accuracy	Correct predictions ÷ all predictions	How often is the prediction correct overall?
Precision	Correct predictions for a class ÷ all predicted as that class	When the model says this class, how often is it right?
Recall	Correct predictions for a class ÷ all true examples of that class	How many real examples of this class did it find?
F1 score	Balance of precision and recall	How well does the model balance both errors?
Macro F1	Average F1 across classes	Does performance remain fair across rare and common classes?

A confusion matrix is a table or heatmap where rows are the true classes and columns are predicted classes. A strong diagonal means many correct predictions. Bright off-diagonal cells show systematic confusion between two labels. This is more useful than accuracy alone because it tells us what the model confuses.

11. Literature Survey and Design Evidence

A literature survey answers two questions: what has already been established by researchers, and how did that evidence influence our implementation? The sources below were checked against the original papers or official documentation. They support design choices; they do not automatically prove that our particular model is optimal.

Source	Main idea	Used here
Stallkamp et al. 2011 — GTSRB	Defines a real-world multi-class traffic-sign benchmark with visual variation.	Dataset choice, 43-class scope, official test evaluation.
Sandler et al. 2018 — MobileNetV2	Efficient inverted residual and depthwise-convolution architecture.	Lightweight transfer-learning comparison model.
Kingma and Ba 2015 — Adam	Adaptive optimizer based on gradient moment estimates.	Optimizer used in the training protocol.
Selvaraju et al. 2017 — Grad-CAM	Gradient-based class-discriminative localization map.	Qualitative explanation and error inspection.
scikit-learn — GroupShuffleSplit	Splits samples by group rather than independently.	Keeps related sequences together during validation.
TensorFlow and Keras documentation	Streaming/prefetched input pipelines, augmentation, and callbacks.	Efficient loading, training-only transformations, checkpointing, and stopping rules.

How to explain the connection: the GTSRB paper motivates the dataset and the need to handle variation; MobileNetV2 motivates a compact comparison architecture; Adam and Keras callbacks define a practical optimization protocol; GroupShuffleSplit protects validation from sequence-related leakage; tf.data keeps the full dataset manageable; and Grad-CAM gives a visual diagnostic. The optional YOLO code is not supported by the GTSRB classification labels, so it is documented as a future detector extension rather than reported as a trained detector.

Presenter wording: "Our literature survey is not a list of unrelated papers. Each reference has a traceable role: benchmark definition, architecture choice, optimizer, leakage control, data pipeline, training control, or interpretability. That traceability makes the implementation easier to justify and reproduce."

Verified reading list: GTSRB primary paper — https://www.ini.rub.de/upload/file/1 470 692 848_f03 494 010c16c36bab9e/StallkampEtAl_GTSRB_IJCNN2011.pdf; MobileNetV2 — https://arxiv.org/abs/1801.04381; Adam — https://arxiv.org/abs/1412.6980; Grad-CAM — https://openaccess.thecvf.com/content_ICCV_2017/papers/Selvaraju_Grad-CAM_Visual_Explanations_ICCV_2017_paper.pdf; GroupShuffleSplit — https://scikit-learn.org/1.5/modules/generated/sklearn.model_selection.GroupShuffleSplit.html; TensorFlow/Keras — https://www.tensorflow.org/guide/data_performance and https://keras.io/api/callbacks/.

12. Actual Results

Model	Input	Run	Accuracy	Macro F1
Custom Cnn	64×64	Colab GPU · 30/25 epochs	97.12%	95.14%
Mobilenet V2	128×128	Colab GPU · 30/25 epochs	76.18%	68.23%

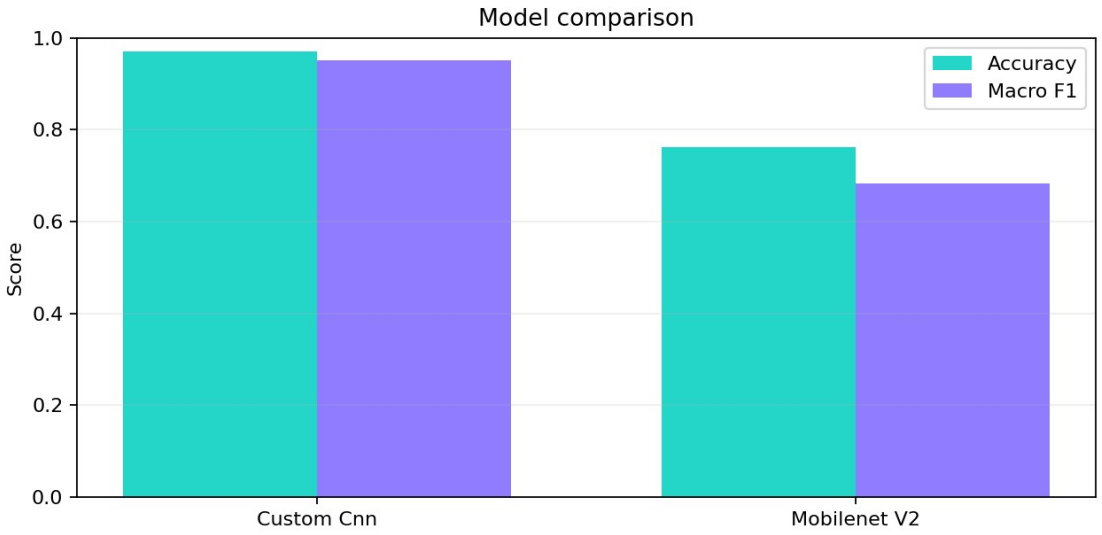


Figure 5. Test accuracy and macro F1.

The Colab GPU run gives the custom CNN 97.12% test accuracy and 95.14% macro F1. MobileNetV2 gives 76.18% test accuracy and 68.23% macro F1. The custom CNN is ahead because it learns directly from the GTSRB images while the frozen MobileNetV2 transfer-learning setup was not fully adapted to this task.

Important qualification: these are reproducible 30/25-epoch baselines, not a maximum-performance claim. Results can vary with the Colab GPU, TensorFlow version, random seeds, and whether class weighting or fine-tuning is enabled. The MobileNetV2 configuration should be improved before treating it as the preferred model.

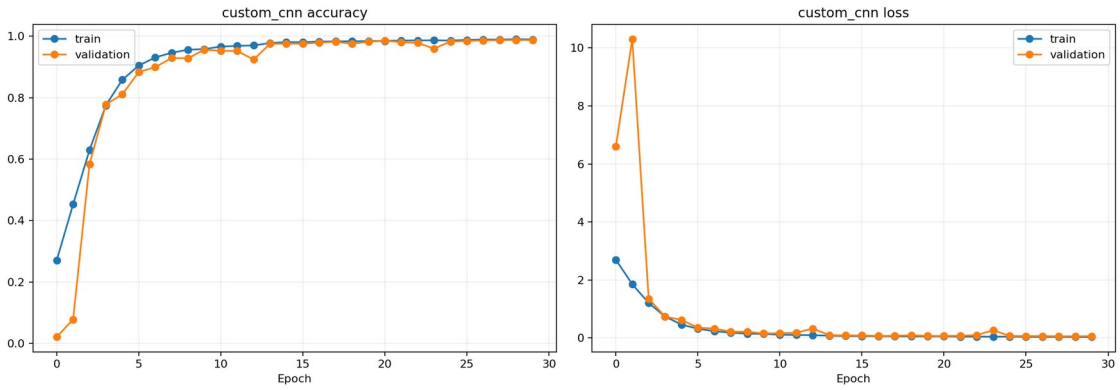


Figure 6. Custom CNN learning curves.

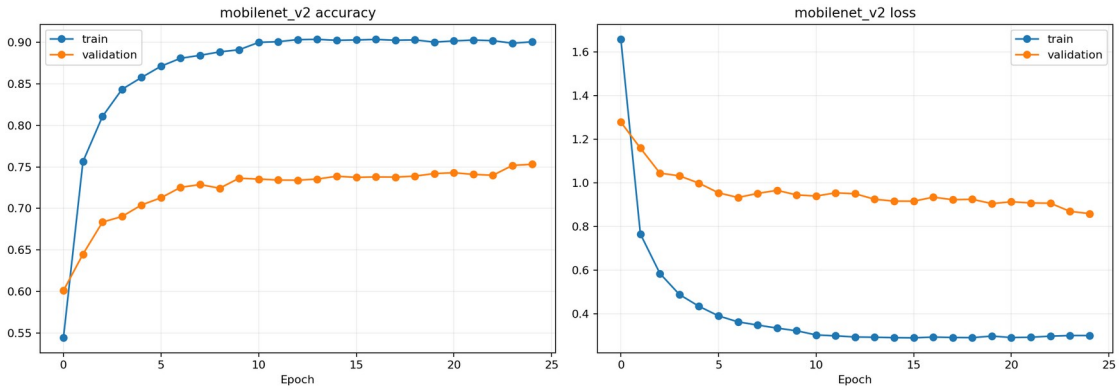


Figure 7. MobileNetV2 learning curves.

Learning curves show how training and validation loss or accuracy change over epochs. If training improves while validation worsens, that is overfitting: the model is memorizing training examples instead of learning patterns that generalize.

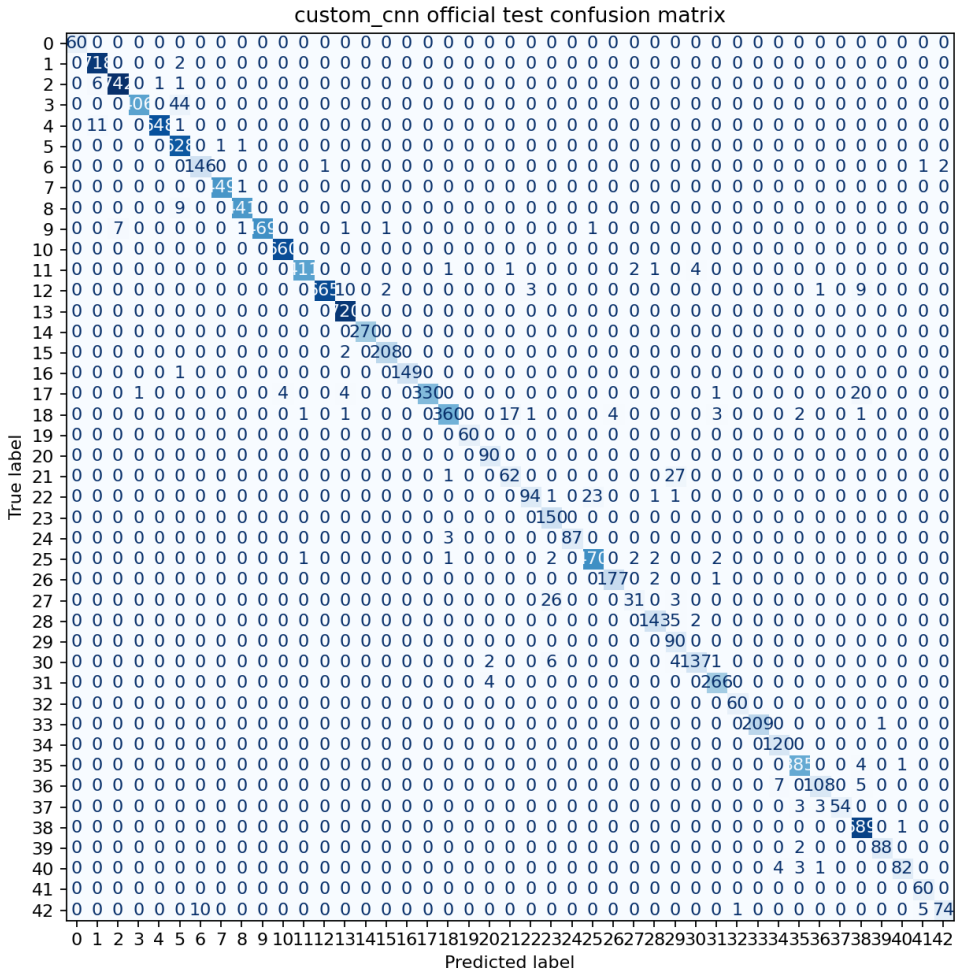


Figure 8. Custom CNN confusion matrix.

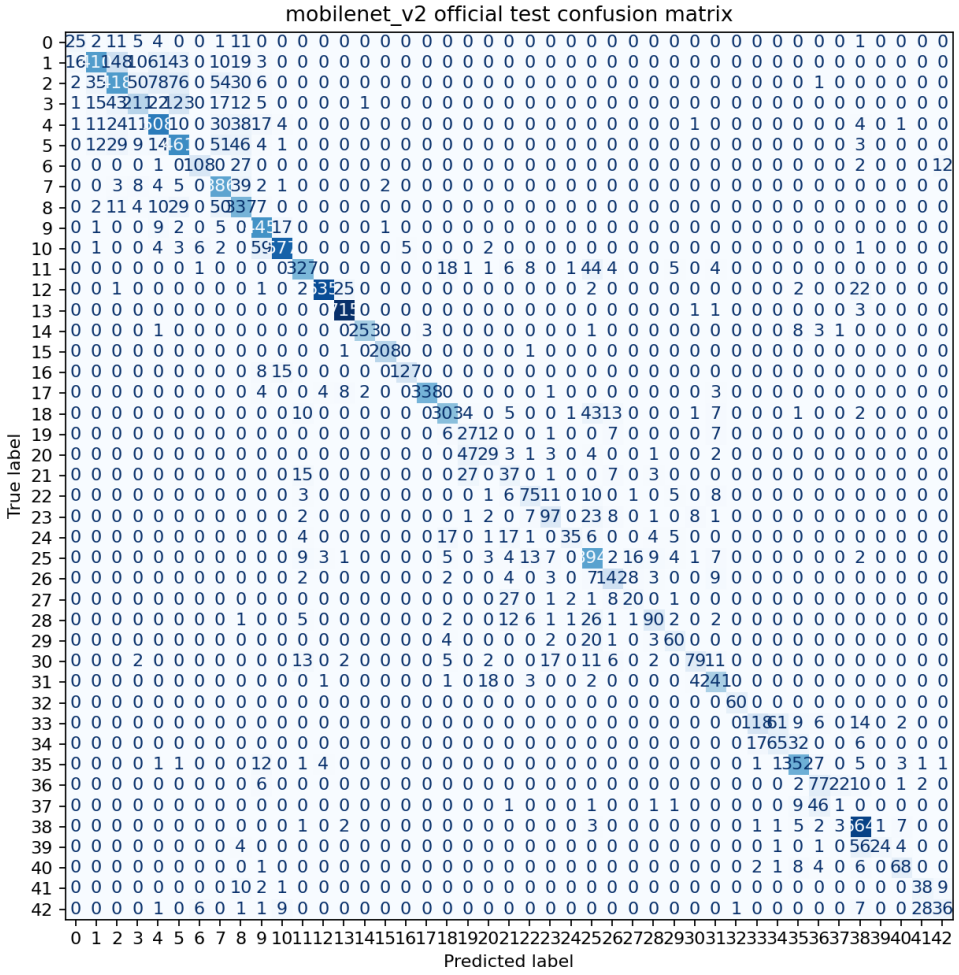


Figure 9. MobileNetV2 confusion matrix.

13. Live Demonstration Script

Use this order during a demo. Explain what each command proves before running it.

Step	Command	What to point out
1. Check data	python scripts/download_dataset.py	Checksum and class validation
2. Generate EDA	python scripts/run_eda.py	Graphs come from the full dataset
3. Train baseline	python -m traffic_sign_recognition.train --model custom_cnn	Model, checkpoint, and history are saved
4. Evaluate	python -m traffic_sign_recognition.evaluate --model-path artifacts/models/custom_cnn/model.keras	Per-class metrics and confusion matrix
5. Predict	python -m traffic_sign_recognition.predict --model-path artifacts/models/custom_cnn/model.keras --image path/to/sign.png	One image becomes a formatted prediction
6. Open dashboard	python -m http.server 8000 --directory site/dist	EDA and results are presented visually

If time is limited, show the already generated graphs and run the prediction command. Full training is compute-heavy; the recorded benchmark in this report was produced on a Colab GPU, while the local M1 Metal setup is also supported for shorter experiments.

14. Questions You May Be Asked

Question	Answer
Why 43 classes?	That is the complete GTSRB classification label space.
Why not use the test set for validation?	Using it for decisions would leak evaluation information and make the final score less trustworthy.
Why report macro F1?	The class distribution is uneven, so macro F1 gives rare classes equal importance.
Why did the custom CNN score higher?	The custom CNN learned directly from GTSRB. MobileNetV2 used a frozen transfer-learning backbone, so it needs fine-tuning for this task.
Is this object detection?	The main pipeline classifies a cropped sign. An optional YOLO scaffold is included for future full-scene detection.
Can it be used for driving decisions?	No. It is an academic classification demonstration and is not safety-certified.

15. Glossary

Word	Meaning
Batch	Small group of examples processed together
Class	One possible category, such as a speed-limit sign
Epoch	One pass over the training data
Feature	A useful visual pattern such as an edge or shape
Generalization	Performing well on unseen examples
Hyperparameter	A setting chosen by the developer, such as learning rate
Label	The correct class attached to an image
Model	A learned function that maps pixels to class scores
Overfitting	Learning the training examples too specifically
Tensor	A numeric array used by the neural network

16. Final Closing Statement

Presenter wording: “This project shows the complete machine-learning lifecycle. We began with a verified dataset, used EDA to understand its challenges, prepared the data carefully, compared two models, evaluated performance with multiple metrics, inspected errors with confusion matrices, and packaged the workflow for reproducible use. The result is a classification system and an honest understanding of what it can and cannot do.”

Remember the three points to emphasize: the data is verified, the test set is kept separate, and the results are interpreted with more than one number.